

```
cmake --build build/  
./build/Debug/task05.exe
```

Aufgabe05

- <https://docs.gl/>
 - <https://learnopengl.com/Advanced-OpenGL/Cubemaps>
-

5.1.0) Erstellen der Cubemap

Erstellen Sie sich die Vertexdaten für einen Würfel. Achten Sie aber darauf, dass sich der Ursprung des Würfels in dessen Schwerpunktmittelpunkt befindet und die Frontfaces des Würfels nach innen zeigen. Schreiben Sie einen Vertex- und Fragmentshader, der Ihnen den Würfel zeichnet.

Implementierung

Wiederverwendung des Würfels aus Aufgabe 03 mit der Änderung, dass die Frontfaces nach innen zeigen. Dafür wurde die Vertex-Winding-Order angepasst, sodass die Vertices im Uhrzeigersinn von außen nach innen definiert werden. Ebenso wurden die Normalenvektoren angepasst, damit sie nach innen zum Ursprung zeigen.

5.1.1) Erstellen der 3D Textur

Erzeugen Sie nun eine neue OpenGL Textur (`glCreateTextures`). Diesmal handelt es sich aber um den Typ: `GL_TEXTURE_CUBE_MAP`. Diese Texturart erlaubt es Ihnen im Shader mit einem 3D-Richtungsvektor von den sechs Seiten der Cubemap zu sampeln. Den Texturspeicher legen Sie mit `glTextureStorage2D` an. Nun können Sie die sechs geladenen Bilder mithilfe der Funktion `glTextureSubImage3D` in den angelegten Speicher der Grafikkarte laden. Nehmen Sie in Ihre Dokumentation mit auf für was die Parameter von `glTextureSubImage3D` stehen und wie Sie vorgegangen sind.

Implementierung

- Zuerst wird mit `glCreateTextures` eine neue Textur vom Typ `GL_TEXTURE_CUBE_MAP` mit einer zuvor erstellten TexturID erstellt.
- In einer For-Schleife werden die sechs Bilder für die sechs Seiten der Cubemap geladen
- Im ersten Durchlauf wird basierend auf der Breite und Höhe des ersten Bildes mit `glTextureStorage2D` der Speicher für die Cubemap-Textur auf der Grafikkarte angelegt. Es wird nur einmal der Speicher angelegt, da alle sechs Bilder die gleiche Breite und Höhe haben müssen. Dies wird durch die OpenGL-Spezifikation für Cubemap-Texturen vorgegeben (gleiche Breite und Höhe für alle Seiten sowie Seiten müssen quadratisch sein).
- Pro Durchlauf der For-Schleife wird mit `glTextureSubImage3D` das aktuell geladene Bild in den zuvor angelegten Speicher der Grafikkarte geladen. Über `zoffset` wird dabei angegeben, auf welche Seite der Cubemap die Bilddaten geladen werden sollen. Die Zuordnung der Seiten erfolgt über den Index `i` der For-Schleife. Index `i` wird dabei in die OpenGL-Enum-Werte (Face-Index) für die Cubemap-Seiten umgerechnet:
 - `i=0` -> `GL_TEXTURE_CUBE_MAP_POSITIVE_X`
 - `i=1` -> `GL_TEXTURE_CUBE_MAP_NEGATIVE_X`
 - `i=2` -> `GL_TEXTURE_CUBE_MAP_POSITIVE_Y`
 - `i=3` -> `GL_TEXTURE_CUBE_MAP_NEGATIVE_Y`
 - `i=4` -> `GL_TEXTURE_CUBE_MAP_POSITIVE_Z`

- i=5 -> GL_TEXTURE_CUBE_MAP_NEGATIVE_Z
- Abschließend werden mit `glTextureParameteri` die Texturparameter für die Cubemap-Textur gesetzt, um ein gewünschtes Verhalten beim Sampling der Textur zu bestimmen.

Frage 1.:

Spielt die Reihenfolge, mit der Sie die einzelnen Bilder hochladen, eine Rolle?

-> **Lösung:**

- Die Reihenfolge beim Hochladen der sechs Bilder spielt in dem Sinne nur eine Rolle, dass sie zum richtigen Enum zugeordnet werden müssen
 - i=0 -> GL_TEXTURE_CUBE_MAP_POSITIVE_X
 - i=1 -> GL_TEXTURE_CUBE_MAP_NEGATIVE_X
 - i=2 -> GL_TEXTURE_CUBE_MAP_POSITIVE_Y
 - i=3 -> GL_TEXTURE_CUBE_MAP_NEGATIVE_Y
 - i=4 -> GL_TEXTURE_CUBE_MAP_POSITIVE_Z
 - i=5 -> GL_TEXTURE_CUBE_MAP_NEGATIVE_Z

```
void glTextureSubImage3D(
    GLuint texture,
    GLint level,
    GLint xoffset,
    GLint yoffset,
    GLint zoffset,
    GLsizei width,
    GLsizei height,
    GLsizei depth,
    GLenum format,
    GLenum type,
    const void *pixels
);
```

- texture: Name/ ID vom Textur-Objekt.
- level: Mipmap-Level der Textur, die aktualisiert werden soll (0 für die Basis-Textur).
- xoffset, yoffset, zoffset: Offset in Pixeln, der angibt, wo die Aktualisierung innerhalb der Textur beginnen soll.
- width, height, depth: Breite, Höhe und Tiefe des Bereichs, der aktualisiert werden soll, in Pixeln.
- format: Format der Pixel-Daten (z.B. GL_RGBA, GL_RGB).
- type: Typ der Pixel-Daten (z.B. GL_UNSIGNED_BYTE).
- *pixels: Zeiger auf die Pixel-Daten.

Wird verwendet, um einen Teil oder die gesamte Textur mit neuen Pixel-Daten zu aktualisieren. In diesem Fall wird es verwendet, um die sechs Bilder in die Cubemap-Textur zu laden.

```
void glTextureParameteri(
    GLuint texture,
    GLenum pname,
    GLint param
);
```

- texture: Name/ ID vom Textur-Objekt.
- pname: Parameter, der gesetzt werden soll (z.B. GL_TEXTURE_WRAP_S, GL_TEXTURE_MIN_FILTER).
- param: Wert, der für den angegebenen Parameter gesetzt werden soll (z.B. GL_CLAMP_TO_EDGE, GL_NEAREST).

Wird verwendet, um Parameter für ein bereits existierendes Textur-Objekt zu setzen.

```
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
```

- Wrap-S -> S steht für die horizontale UV-Koordinate.
- Definiert den Fall, wenn UV-Koordinaten außerhalb des Bereichs [0, 1] liegen.
GL_CLAMP_TO_EDGE sorgt dafür, dass für alle UV-Koordinaten außerhalb des Bereichs die äußerste Randfarbe der Textur verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

- Wrap-T -> T steht für die vertikale UV-Koordinate.
- Definiert den Fall, wenn UV-Koordinaten außerhalb des Bereichs [0, 1] liegen.
GL_CLAMP_TO_EDGE sorgt dafür, dass für alle UV-Koordinaten außerhalb des Bereichs die äußerste Randfarbe der Textur verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
```

- Min-Filter -> Definiert die Filtermethode, die verwendet wird, wenn die Textur verkleinert dargestellt wird (d.h. wenn mehrere Texel auf einen Pixel abgebildet werden). GL_NEAREST sorgt dafür, dass der nächstgelegene Texel-Wert verwendet wird.

```
glTextureParameteri(textureHandle, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
```

- Mag-Filter -> Definiert die Filtermethode, die verwendet wird, wenn die Textur vergrößert dargestellt wird (d.h. wenn ein Texel auf mehrere Pixel abgebildet wird). GL_NEAREST sorgt dafür, dass der nächstgelegene Texel-Wert verwendet wird.

5.1.2) Samplen von der Cubemap Textur

- Im Vertexshader übergibt man die `in_Position` der Vertices im Objektkoordinatensystem als Texturkoordinaten an den Fragmentshader.
- Im Fragmentshader sampelt man mit `texture(skyboxCubemap, in_TexCoord)` von der Cubemap-Textur. `in_TexCoord` ist dabei die Objektkoordinaten der Vertices. Durch die Zentrierung des Würfels um den Ursprung (Kamera) und einer ViewMatrix ohne Translation (siehe 5.3), entsprechen die Objektkoordinaten der Vertices gleichzeitig den Richtungsvektoren von der Kamera zu den Vertices. Somit kann man die Objektkoordinaten der Vertices direkt als Texturkoordinaten für das Sampling der Cubemap-Textur verwenden. Das Samplen von der Cubemap-Textur mit einem Richtungsvektor ermöglicht es, die entsprechende Seite der Cubemap zu verwenden und die korrekte Farbe basierend auf der Richtung zu liefern.

5.1.3) Kamera

Implementieren Sie Tastaturinput, sodass Sie mit der Kamera in der Szene herumfahren und rotieren können.

Implementierung

- Wie in der vorherigen Aufgabe 03 bereits implementiert, wird die Kamera über die WASD-Tasten für die Translation und die Pfeiltasten für die Rotation, bzw. QE-Tasten für die Rotation um die Blickrichtung (Roll), gesteuert. Für die Kamerabewegung werden die internen Funktionen der Kamera-Klasse verwendet, um die Variablen der Kamera entsprechend zu aktualisieren und danach mit diesen die ViewMatrix zu berechnen.
-

5.2) Environment Mapping

Sie können die Cubemap dafür nutzen, um (perfekte) Reflexionen an einem Modell zu simulieren. Ermöglichen Sie dem neuen Shaderpaar ebenfalls auf die Cubemap Textur zuzugreifen. Lassen Sie nun die Cubemap an Ihrem Modell spiegeln. Nutzen Sie dafür die GLSL Funktion `reflect`.

Implementierung

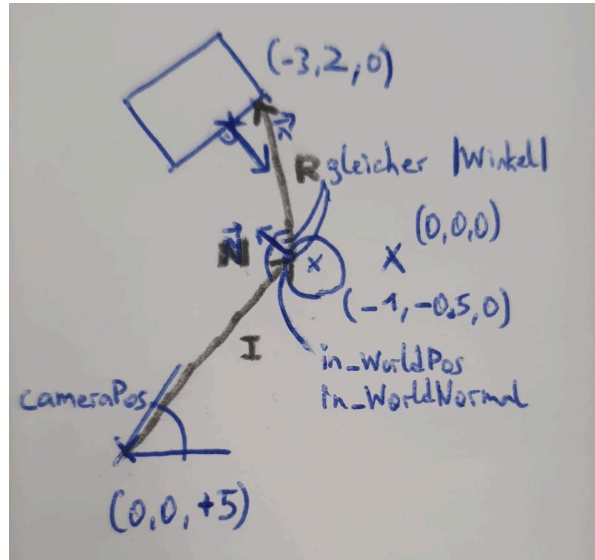


Figure 1: Zeichnung der Umgebungsmapping über berechneten Reflektionsvektor.

- Für das Umgebungsmapping wird ein neuer Shader geschrieben, der zusätzlich Zugriff auf die Cubemap-Textur hat, um die mithilfe des berechneten Reflektionsvektors die entsprechende Farbe von der Cubemap zu sampeln.

Vertex-Shader:

- Im Vertexshader werden die Vertex-Positionen und Normalenvektoren vom Objektkoordinatensystem in das Weltkoordinatensystem transformiert (`u_ModelMat`) und an den Fragmentshader übergeben.

Fragment-Shader:

- Im Fragmentshader werden zusätzlich die Weltkoordinaten der Kamera über eine Uniform (`u_CameraPos`) übergeben.
- I ist der Richtungsvektor von der Kamera zum Fragment des Objekts, worauf die Reflektion abgebildet werden soll. Den Richtungsvektor (einfallender Strahl vom Auge) wird durch die Subtraktion der Weltkoordinaten der Kamera von den Weltkoordinaten des Fragments berechnet.
- N ist der Normalenvektor am Fragment, der ebenfalls in Weltkoordinaten vorliegen muss.
- R ist der berechnete Reflektionsvektor, der mit der GLSL-Funktion `reflect(I, N)` berechnet wird. Diese Funktion berechnet den Reflektionsvektor basierend auf dem einfallenden Richtungsvektor I und dem Normalenvektor N .
 - `reflect` spiegelt I an der Ebene, die durch N definiert ist (Einfallswinkel == Ausfallswinkel). Es wird über $\text{dot}(N, I)$ die Projektion von I auf N berechnet und diese doppelt von I abgezogen, um eine Spiegelung zu erreichen — $I - 2 \cdot \text{dot}(N, I) \cdot N$.
 - Das Ergebnis R ist der Reflektionsvektor, der die Richtung angibt, in die das Licht reflektiert wird.
- Anschließend wird mit `texture(u_Cubemap, R)` von der Cubemap-Textur gesampelt, um die Farbe zu erhalten, die an diesem Fragment reflektiert wird. Dabei ist R ein 3D-Richtungsvektor der neben den UV-Koordinaten auch die richtige Seite der Cubemap auswählt, von der gesampelt

werden soll, indem die Komponente mit der größten absoluten Ausprägung von R die Cubemap-Seite bestimmt und die anderen beiden Komponenten von R als UV-Koordinaten für das Sampling verwendet werden.

5.3) FPS Camera

Ändern Sie nun das Verhalten der Cubemap so, dass Sie diese nicht mehr verlassen können! Dies soll den Eindruck eines unendlich entfernt liegenden Horizonts simulieren, wie es auch in vielen Videospielen der Fall ist.

Implementierung

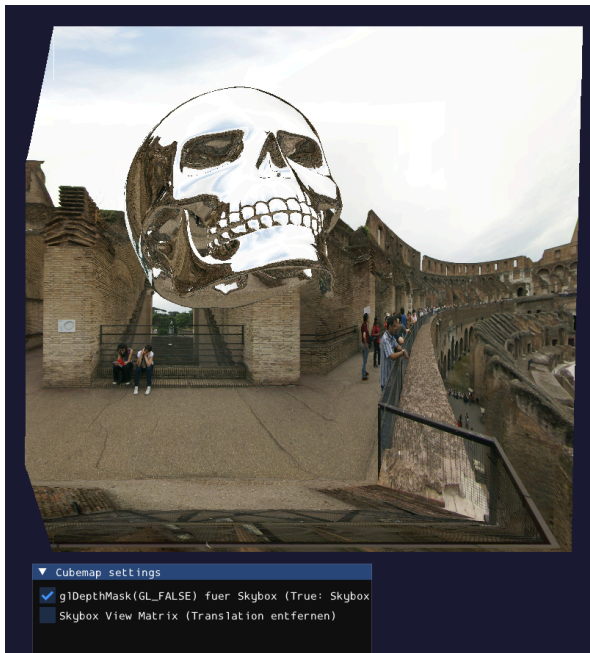
```
glDepthMask(GL_FALSE);
```

- Deaktiviert das Schreiben in den Tiefenpuffer. Dadurch wird verhindert, dass die Cubemap-Geometrie die Tiefeninformationen überschreibt, was wichtig ist, um sicherzustellen, dass andere Objekte korrekt vor der Cubemap gerendert werden können.

```
Mat4f skyboxViewMat = viewMat;  
if (useSkyboxViewMat) {  
    skyboxViewMat[3][0] = 0.0f;  
    skyboxViewMat[3][1] = 0.0f;  
    skyboxViewMat[3][2] = 0.0f;  
}
```

- Modifiziert die View-Matrix, um die Translationskomponenten zu entfernen. Dadurch bleiben die Objekte, wie die Cubemap, immer um den Ursprung zentriert, unabhängig von der Position der Kamera, da die Verschiebung/ Translation auf Null gesetzt wird. Im Kamerakoordinatensystem ist die Kamera immer im Ursprung, und die Cubemap wird um diesen Ursprung herum gerendert, wodurch der Eindruck eines unendlich entfernten Horizonts entsteht.
 - Wird die Kamera bewegt, würde normalerweise die View-Matrix eine Translation enthalten, die die Position der Kamera berücksichtigt. Durch das Setzen der Translationskomponenten auf Null wird diese Translation entfernt, und die Cubemap bleibt immer um den Ursprung zentriert, wodurch der Effekt eines unendlich entfernten Horizonts erzielt wird.
-

Ergebnis



(a) Mit Kameratranslation; Cubemap bewegt sich mit der Kamera und verlässt damit den Ursprung (Kamera)



(b) Ohne Kameratranslation; Cubemap bleibt immer um den Ursprung (Kamera) zentriert

Figure 2: Vergleich der beiden Ergebnisse mit und ohne ViewMat Translation der Skybox